MARMARA UNIVERSITY
Artificial Intelligence in Health Science

Whole-cell segmentation of tissue images using large-scale data annotation and deep learning

Yunus Emre Kılıçkıran 150720053

Submission Date: Dec 31, 2024
Instructure: Prof. Gökhan Bora Esmer

Purpose

- Understand the machine learning techniques for images
- To solve the cell-segmentation problem

Problem Definition and Related Work

Accurately segmenting cells in tissue images is critical for understanding structural and functional relationships within tissues, a cornerstone of translational research. However, cell segmentation remains a persistent challenge due to variability in cell shape, size, and density across different tissue types. Existing machine-learning approaches often fail to generalize without manual, image-specific adjustments, limiting scalability in large-scale tissue imaging studies. The lack of substantial annotated datasets for whole-cell segmentation further exacerbates the problem, as most available datasets focus on nuclear segmentation. This limitation hinders the development of algorithms capable of analyzing comprehensive cell morphology and subcellular localization. Addressing these challenges requires scalable, automated solutions that achieve human-level accuracy while minimizing the burden of manual annotation.

In the provided article tackles challenges in cell segmentation by introducing TissueNet and Mesmer, which combine large-scale data annotation with deep learning. TissueNet is a large dataset containing over 1 million labeled cells, much larger than previous datasets. It includes diverse tissue types, imaging platforms, and both healthy and diseased samples. The dataset was created using a human-in-the-loop process, where experts and crowd annotators worked together with an improving model to reduce manual effort over time.

Using TissueNet, the researchers developed Mesmer, a deep-learning algorithm for nuclear and whole-cell segmentation. Mesmer achieved human-level accuracy (F1 score of 0.82) and works well across different tissues and imaging methods without manual adjustments.

To make Mesmer widely accessible, the team integrated it into the DeepCell platform, offering both cloud and local tools for researchers. Mesmer supports detailed analyses like protein localization and cell morphology profiling, enabling insights that were difficult with previous methods. This solution simplifies large-scale tissue imaging analysis and makes cell segmentation more accurate and efficient.

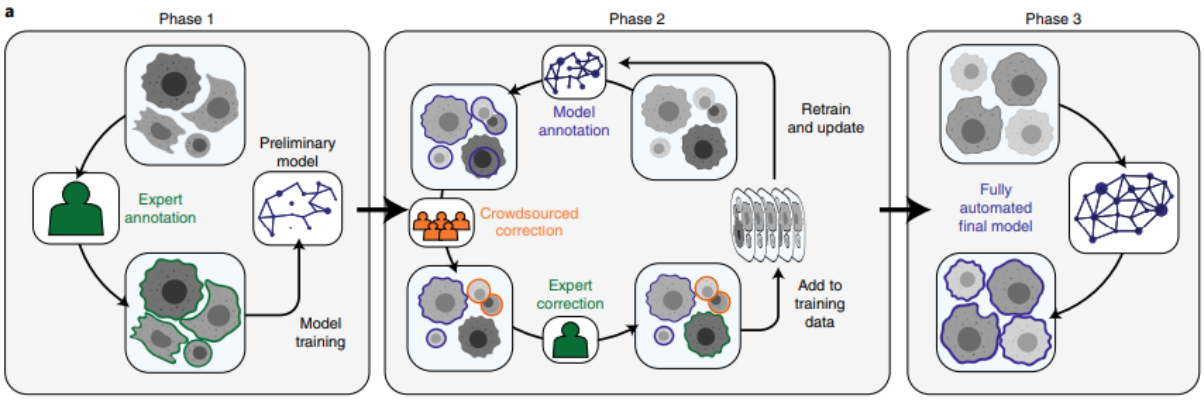

Figure 1- Method of the provided article

Methodology

A. Data-Preprocessing

i. Data Usage Technique

In this project, I used the TissueNet dataset, which contains three .npz files designated for training, testing, and validation. Each file includes compressed image arrays stored as X and y keys. The X key contains raw, unannotated images, while the y key holds pre-annotated images. Additionally, the y key includes two channels: one for cell segmentation and the other for nuclear segmentation.

In the initial stages of my work, I attempted to load the .npz file directly into memory to train the model. However, due to the large size of the dataset, I encountered memory errors, as my system was not capable of handling the entire dataset at once. To address this issue, I tried various approaches, such as splitting the data, implementing lazy loading to load the data sequentially, and reducing the dataset size. Unfortunately, none of these solutions provided a permanent fix.

To better understand whether the issue was with the dataset, my system, or the images themselves, I attempted to train the model using just a single image. Surprisingly, I still encountered memory errors. This led me to conclude that the problem lay in how the images were processed. Specifically, I realized that the model was being trained using high-dimensional arrays, which posed significant challenges for memory management.

To resolve this issue, I wrote a code to convert each array into individual images and saved them into directories. By transforming the entire dataset into images and storing them in folders, I was able to bypass the memory limitations. Subsequently, during training, I loaded these images sequentially into the model. This approach successfully eliminated the memory error and allowed me to proceed with training without interruptions.

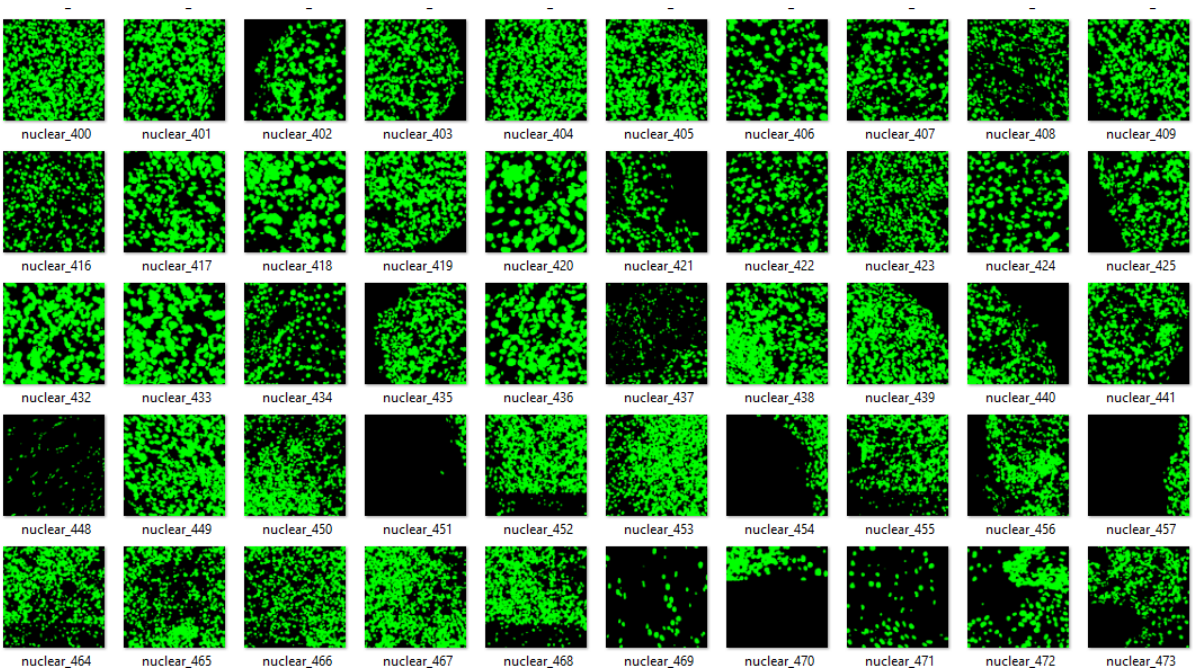

Figure 2- Image dataset

ii. Resizing Images

All input images are fixed to target size of 128x128 pixels using the resize method. This ensures uniformity in the input dimensions, which is crucial for the model to process the data effectively. By standardizing the image dimensions, the model avoids issues related to varying image sizes, simplifying training and improving performance.

iii. Normalization

All image pixel values are normalized to a range between 0 and 1 by dividing by 255. This step helps the model converge faster by ensuring that input features are on a similar scale. Normalization also prevents large values from dominating the gradients during backpropagation, enhancing numerical stability.

iv. Batching

The dataset is processed in batches of size 8, as specified by the BATCH_SIZE variable. Batching helps to optimize memory usage and speeds up training by processing multiple images in parallel.

v. Prefetching

The model uses prefetching to overlap the data loading and preprocessing steps with model training. This improves overall performance by reducing the time spent waiting for data to be prepared during training.

vi. Conversion to Grayscale

Images are converted to grayscale. This simplifies the input data by reducing the dimensionality and focuses the model's attention on intensity rather than color, which is appropriate for many segmentation tasks.

B. Model Training

During the training phase of the model, my initial literature review revealed that various methods were used for image segmentation tasks. One of these methods was the U-Net model. U-Net is a convolutional neural network (CNN) architecture specifically designed for pixel-level image segmentation tasks, such as biomedical image analysis. It is particularly effective due to its encoder-decoder structure. The encoder extracts features from the input image using convolutional and max-pooling layers, progressively reducing the spatial dimensions while increasing feature complexity. The decoder then reconstructs the spatial dimensions using up-sampling and concatenation operations, ensuring that both global context and fine details are preserved.

This model is well-suited for tasks requiring precise segmentation, such as nuclear and cell segmentation in this project. Its ability to leverage skip connections between the encoder and decoder allows the model to retain low-level features, making it ideal for identifying sharp boundaries and intricate structures. Additionally, U-Net has proven effective with small and medium-sized datasets, as it can produce accurate results even with limited data.

Given its widespread success in biomedical segmentation and its efficiency in handling pixel-level annotations, U-Net was selected as the primary model for this project. Its ability to generate detailed and accurate segmentation masks made it a fitting choice for solving the challenges of this task.

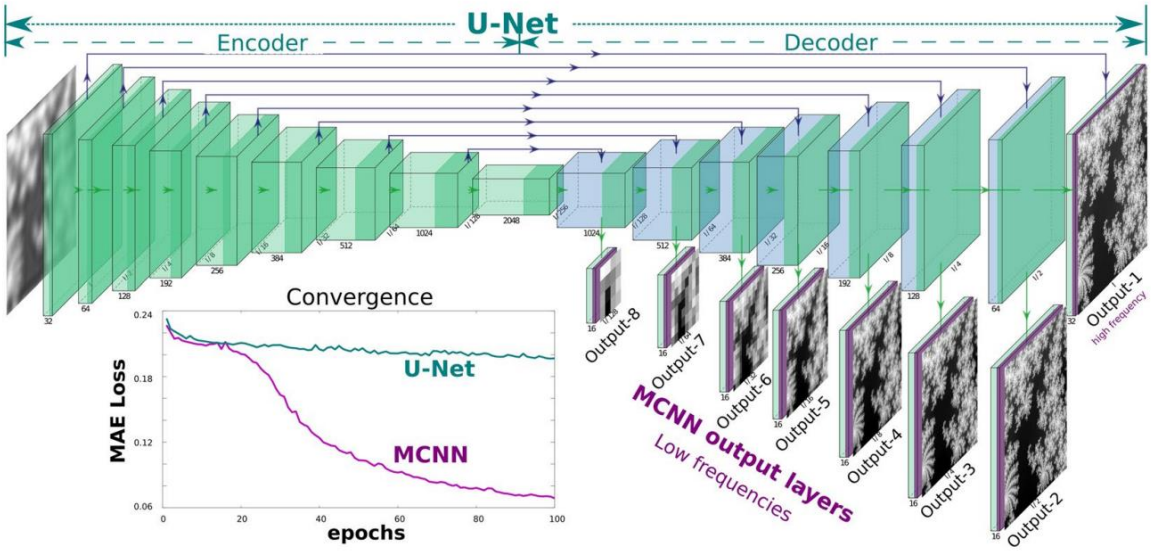

Figure 3- U-Net working mechanism

- Step 1: Model Compilation

Before training begins, the U-Net model is compiled using the Adam optimizer, which is effective for tasks involving large datasets and non-convex optimization problems. The learning rate is set to 1e-4, ensuring that the model learns at a controlled pace without overshooting the optimal solution. The binary cross-entropy loss function is chosen to quantify the difference between predicted and true segmentation masks, as this task involves binary classification at the pixel level. Accuracy is included as a performance metric to monitor how well the model predicts the correct segmentation.

- Step 2: Training Dataset

The preprocessed training dataset, consisting of resized and normalized images and masks, is fed into the model in batches of size 8. Batching allows for efficient memory usage and parallel computation. The tf.data API ensures that data loading and preprocessing are optimized with lazy loading, reducing memory constraints by processing only the required data at each step.

- Step 3: Epochs and Iterations

The model is trained over 10 epochs. An epoch represents one full pass through the entire training dataset. Within each epoch, the dataset is divided into smaller batches, and the model updates its weights iteratively after processing each batch. This iterative process helps the model gradually learn the mapping between input images and corresponding masks.

- Step 4: Validation During Training

During training, the validation dataset is used to evaluate the model's performance after each epoch. Validation helps monitor overfitting by assessing how well the model generalizes to unseen data. The validation loss and accuracy are calculated, providing feedback on the model's progress and helping determine whether adjustments are needed to improve performance.

- Step 5: Weight Updates

As each batch of data is processed, the model calculates the gradients of the loss function with respect to its weights using backpropagation. These gradients are used by the Adam optimizer to update the weights, minimizing the loss function and improving the model's predictions in the next iteration.

- Step 6: Feedback and Adjustments

The training process is guided by metrics such as training loss, validation loss, training accuracy, and validation accuracy. If validation performance deteriorates while training accuracy improves, it signals overfitting, which could prompt adjustments such as reducing the learning rate, adding regularization, or increasing dropout.

- Outcome of the model

At the end of the training phase, the model has learned to map input images to segmentation masks effectively. The weights of the model are optimized to minimize the loss, ensuring accurate predictions for the given task. The model is then ready to be evaluated on the test dataset and used for generating segmentation masks in real-world scenarios.

C. Results and Discussion

In this section, I present the outcomes of training the U-Net model for image segmentation and discuss the observations made during the process. To optimize the model's performance, I experimented with various hyperparameters, including the learning rate, number of epochs, and data size. By analyzing the results of these configurations, I observed how each hyperparameter affected the model's ability to learn and generalize. Additionally, I evaluated the model's predictions on the test set and visualized its segmentation performance to assess its accuracy and robustness. The following subsections provide a detailed analysis of the results, comparing the model's performance under different settings and highlighting the trade-offs encountered.

In this analysis, we will use three key metrics—F1 score, Intersection over Union (IoU), and loss—to evaluate the model's performance in the segmentation task. The F1 score balances precision and recall, providing a comprehensive measure of the model's ability to correctly classify positive pixels. IoU measures the overlap between the predicted segmentation mask and the ground truth, offering insight into the spatial alignment of predictions. Loss, calculated using binary cross-entropy, quantifies the error between predictions and actual segmentation masks, guiding the optimization process. Together, these metrics provide a detailed understanding of the model's performance and effectiveness.

i. Adjusting epoch number

Adjusting the number of epochs is important because it determines how many times the model will process the entire dataset during training. This allows the model to gradually learn the relationship between the input images and their corresponding segmentation masks. By providing the model with sufficient epochs, it can better capture the underlying features necessary for accurate segmentation.

Number Of Epochs	**F1 Score**	**Loss**	**Intersection over Union (IoU)**
1	0.50	0.503	0.337
5	0.66	0.327	0.493
10	0.71	0.267	0.55

The table summarizes the model's performance metrics—F1 score, loss, and Intersection over Union (IoU)—across different numbers of epochs (1, 5, and 10). These results provide insights into how extended training influences segmentation accuracy and model optimization.

The F1 score starts at 0.50 after 1 epoch, indicating the model's initial ability to balance precision and recall. After 5 epochs, the F1 score increases to 0.66, showing significant improvement in correctly identifying positive pixels. By 10 epochs, the F1 score reaches 0.71, demonstrating that extended training enables the model to better classify positive and negative pixels, leading to more accurate segmentation results.

The loss decreases consistently as the number of epochs increases, starting at 0.503 after 1 epoch and dropping to 0.267 by 10 epochs. This steady reduction indicates that the model effectively minimizes the error between predicted and actual segmentation masks over time. The decreasing loss reflects improved convergence and more precise predictions with extended training.

The IoU starts at 0.337 after 1 epoch, highlighting moderate overlap between the predicted and ground truth segmentation masks. After 5 epochs, IoU improves to 0.493, and by 10 epochs, it reaches 0.55, signifying that the model achieves better spatial alignment with extended training. The increasing IoU demonstrates the model's growing ability to produce masks that closely match the ground truth.

The results show that increasing the number of epochs significantly enhances the model's segmentation performance. The consistent improvement in F1 score and IoU, coupled with the reduction in loss, highlights the model's ability to learn more accurate and meaningful patterns with extended training. However, the diminishing rate of improvement between 5 and 10 epochs suggests that the model may be nearing convergence. Additional epochs may yield marginal gains and should be assessed for trade-offs between performance and training time.

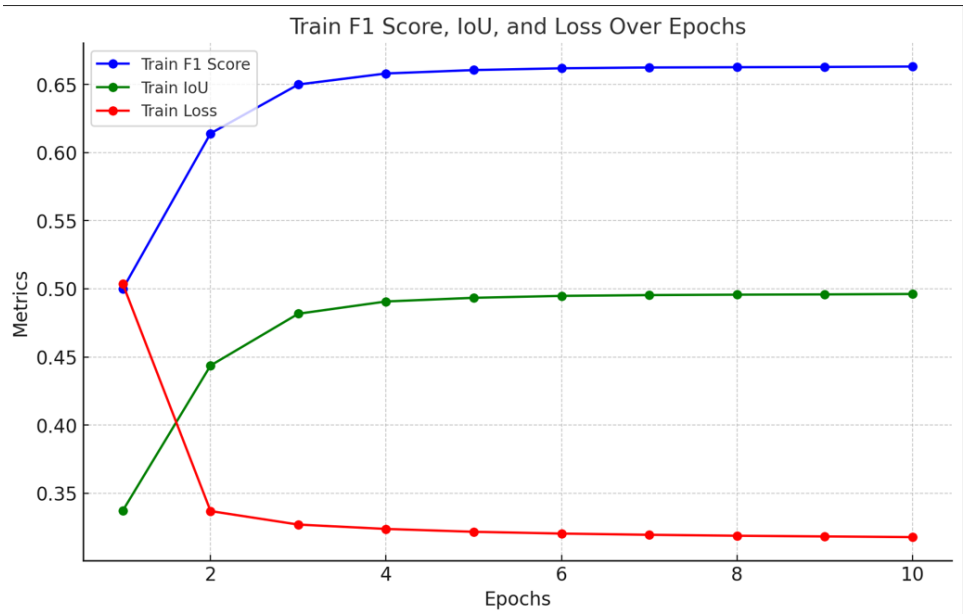

Figure 4- change of matrices over the epoch number

ii. Adjusting Data size

Adjusting the size of the dataset is important to ensure the model is exposed to sufficient information for learning while remaining manageable within computational constraints. It allows us to determine the optimal amount of data required for the model to effectively capture patterns without redundancy or resource inefficiency. Exploring different data sizes helps balance the need for diverse input with practical training considerations.

Number of Data Image	**F1 Score**	**Loss**	**Intersection over Union (IoU)**
700	0.0001	0.188	$4x10^{-12}$
1900	0.30	0.191	0.18
3500	0.71	0.19	0.548

The results indicate how the performance metrics, specifically F1 score, loss, and Intersection over Union (IoU), vary as the dataset size increases from 700 to 3500 images. These metrics offer valuable insights into the model's learning behavior and generalization capabilities.

The F1 score starts at an extremely low value when trained on 700 images, suggesting that the model is unable to balance precision and recall effectively with a small dataset. As the dataset size increases to 1900 images, the F1 score rises significantly to 0.30, indicating that the model is beginning to identify patterns and correctly classify positive pixels. With 3500 images, the F1 score improves further to 0.71, demonstrating that the larger dataset provides the model with more diverse and meaningful information, enabling better segmentation performance.

The loss remains relatively stable across all dataset sizes, with minor fluctuations. At 700 images, the loss is 0.188, and it increases slightly to 0.191 with 1900 images, before settling at 0.19 with 3500 images. The consistent loss values suggest that the model converges adequately

regardless of the dataset size. However, the relatively high loss with smaller datasets might reflect the difficulty the model faces in learning from limited data.

The IoU starts at an exceptionally low value for 700 images, indicating that the predicted masks have almost no overlap with the ground truth masks. This is consistent with the poor F1 score observed at this stage. With 1900 images, the IoU improves to 0.18, and with 3500 images, it reaches 0.548, showing substantial improvement. The higher IoU with the largest dataset highlights the model's enhanced ability to generate segmentation masks that align with the ground truth.

The results highlight the importance of dataset size in training a segmentation model. With 700 images, the model struggles to generalize, as shown by low F1 score and IoU. Increasing to 1900 images improves variability and metrics, while 3500 images achieve the best performance due to increased diversity and information. The stable loss across datasets suggests effective learning without overfitting, aided by regularization. However, further improvements in data diversity, class balance, and architecture could enhance segmentation performance and generalization.

C. Future Work and Improvements

The outcomes of this project demonstrated significant progress in whole-cell segmentation using deep learning, yet several avenues for future work and improvements remain open.

1. **Expanding the Dataset**
   While the current dataset includes diverse tissue types and imaging methods, expanding it further to encompass more rare tissue types and pathological conditions can enhance the model's generalization capabilities. Incorporating augmented datasets could also help address the limitations posed by underrepresented classes in the training data.

2. **Implementing Advanced Architectures**
   Although U-Net performed well in this task, exploring other state-of-the-art architectures such as Vision Transformers or attention-based mechanisms could potentially improve segmentation precision and boundary sharpness. Additionally, integrating ensemble models might offer robust performance across varied datasets.

3. **Automated Hyperparameter Optimization**
   Manual tuning of hyperparameters such as learning rate and batch size provided insights, but using automated optimization techniques like Bayesian optimization or grid search could yield more optimal configurations, improving both accuracy and convergence speed.

CONCLUSION

This study explored the application of deep learning for cell segmentation in tissue images, a critical task in biomedical research and diagnostics. Leveraging the TissueNet dataset and the U-Net architecture, the project achieved promising results, demonstrating the effectiveness of using large-scale annotated datasets and robust model architecture for segmentation tasks.

The methodology addressed challenges such as memory limitations and variability in data input by employing techniques like sequential data loading, normalization, and image resizing. The training process highlighted the importance of hyperparameter tuning, dataset size, and

iterative evaluation, leading to steady improvements in performance metrics, including F1 score, Intersection over Union (IoU), and loss reduction. The visualized results further validated the model's ability to generalize and align segmentation masks closely with the ground truth.

Despite the advancements, certain limitations remain, including challenges in boundary precision and the need for broader generalization across diverse datasets. Future work involving expanded datasets, advanced architectures, and optimization techniques could further enhance segmentation accuracy and applicability in real-world scenarios.

Overall, this project underscores the potential of deep learning in addressing complex challenges in biomedical imaging. By building on these findings and integrating proposed improvements, this work contributes to the development of scalable, efficient, and precise tools for tissue analysis, enabling deeper insights into cellular morphology and function in both research and clinical settings.